



Volume Number: 13 (1997)

Issue Number: 6

Column Tag: Programmer's Challenge

June 1997 Programmer's Challenge

by Bob Boonstra, Westford, MA

Turing Machine

A Turing Machine is a finite state machine augmented with an infinite amount of external storage, formatted as a sequence of squares on a linear tape. The behavior of the Turing Machine is described by a set of rules, each of which contains a current state, an input symbol, a next state, an output symbol, and a move direction. At any given time, the Turing Machine is described by the current internal state, the position of its read-write head on the tape, and the contents of the tape. Each clock cycle the Turing Machine reads the square of the input tape on which the read-write head is positioned, replaces the contents of that square by writing an output symbol based on the input and the current state, and moves left or right one square on the tape (or halts). Because they have access to an unbounded amount of storage, Turing Machines can solve problems that finite state machines cannot. An example of such a problem is parenthesis checking. Given an input of left and right parentheses, delimited by the special symbol 'A', such as:

0000A(((((((A0000

...the following set of rules will determine whether the parentheses are well-formed, meaning that they can be paired off so that each left parenthesis has a balancing right parenthesis:

State	Input	NewState	Output	Move
0	)	1	X	Left
0	(	0	(	Right
0	A	2	A	Left
0	X	0	X	Right
1	)	1	)	Left
1	(	0	X	Right
1	A	1	0	Halt
1	X	1	X	Left
2	(	2	0	Halt
2	A	2	1	Halt
2	X	2	X	Left

This machine scans right in state 0 looking for a right parenthesis. When it finds one, it moves to state one, replaces the right parenthesis with an 'X', and moves to state 1 to scan for a left parenthesis., which it also replaces with an 'X'. If it encounters the 'A' delimiter when looking for a balancing parenthesis, it halts after writing a '0', indicating that the parentheses are not well-formed. If all parentheses are paired off, it writes a '1' indicating the input is well-formed.

Your Challenge this month is to implement a Turing Machine. The prototype for the code you should write is:

```
typedef unsigned long ulong;

typedef enum {kMoveLeft=-1,kHalt=0, kMoveRight=1} MoveDir;

typedef struct TMRule { /* rule in program for TM */
    ulong oldState;      /* this rule fires when currentState == oldState and */
    ulong inputSymbol;   /* currentSymbol == inputSymbol */
    ulong newState;      /* set currentState to newState when this rule fires */
    ulong outputSymbol;  /* write outputSymbol to tape when this rule fires */
    MoveDir moveDirection; /* move left or right as indicated when this rule fires */
} TMRule;
```

Showcase

CAPE AND TIGHTS NOT INCLUDED

Switch from reactive to **PROACTIVE** support and be the superhero your users expect

WATCHMAN
 MONITORING

THIS MONTH'S ISSUE



Issue 353

Code Debugging and Spelunking

Check out the latest including: x86 Code Debugging and Spelunking ; An Introduction to WebGL ; Managing ESXi from your Mac ; Deploying SHA-3 ; An Introduction to Julia ; KoolTools: BBEdit 11.5, TextSoap For OS X, Honeywell Lyric Round Wi-Fi Thermostat, My Cloud EX2 Ultra ; MacTech Spotlight: Daniel H Steinberg

[See Now](#)


TODAY'S DEAL

```

typedef void (*TMMoveProc) (
    ulong outputSymbol,
    ulong newState,
    MoveDir moveDirection
);

Boolean /* success */ TuringMachine(
    const TMRule theRules[], /* pointer to program for TM */
    ulong numRules,          /* number of rules in TM program */
    ulong *theTape,          /* pointer to input tape for TM */
    ulong tapeLen,           /* theTape[0]..theTape[tapeLen-1] is valid */
    long rwHeadPos,          /* TM read head is at theTape[rwHeadPos] */
    TMMoveProc ReportMove    /* callback proc to inform caller of each move */
);

```

On input, your **TuringMachine** will be provided with **numRules** rules governing the behavior of the Turing Machine. The rules are pointed to by **theRules**. You will also be provided with a input tape pointed to by **theTape**, with a finite number of contiguous squares preinitialized to the Turing Machine input. The read-write head will be positioned on the input tape at **theTape[rwHeadPos]**. All other squares of the input tape will be empty, indicated by a binary zero. Your **TuringMachine** should begin in state 0, read the first input symbol, and find the appropriate rule. It should invoke the callback **ReportMove** to inform my test code of what your machine is doing, providing the **outputSymbol** that you will write to the tape, the **newState** of your finite state machine, and the **moveDirection** for the new position of the read-write head. You should then update **theTape**, move your read-write head, and update the Turing Machine state.

When your **TuringMachine** encounters a rule with a **moveDirection** of **kHalt**, you should make a final callback to **ReportMove** and then return **TRUE**. If your TuringMachine exceeds the **tapeLen** of **theTape**, or if you are unable to find a rule that applies to your current machine state and the input symbol, you should return **FALSE**. It is my intent to provide all necessary input rules and a sufficient length of tape, but your machine should fail gracefully if an input bug or an implementation bug results in running out of tape or encountering a bad input symbol.

Your code will be timed with a sequence of rule/tape pairs, and the solution that completes execution of the inputs correctly in the shortest total time will be the winner. Half of the tests will use a "universal" Turing Machine, where the rules describe a general Turing Machine interpreter, and the input tape contains another Turing Machine program. The time executed by the **ReportMove** callback will be included in your solution time. The callback will look something like the following.

```

static void ReportMove(long outputSymbol, long newState, MoveDir moveDirection)
{
    if ((gTapePos>=0) && (gTapePos<gTapeLen))
        gTape[gTapePos] = outputSymbol;
    gTapePos += moveDirection;
    gState = newState;
}

```

You may not modify the input rules pointed to by **theRules**, but you may allocate reasonable additional storage if you would like to preprocess the rules in some way, provided you deallocate the storage before returning.

For those of you that would like additional information on Turing Machines, you can refer to almost any textbook on automata theory or the theory of computation. My personal favorite is Computation, Finite and Infinite Machines, by Marvin Minsky, © 1976 by Prentice-Hall.

This will be a native PowerPC Challenge, using the latest CodeWarrior environment. Solutions may be coded in C, C++, or Pascal.

Three Months Ago Winner

Congratulations to **Gregory Cooper** for narrowly defeating the second place entry by **Jeff Mallett** in the March Hex Challenge. Hex is played on an NxN rhombus of hexagons, with players alternately occupying hexagons, one playing vertically and the other horizontally, each trying to complete an unbroken chain before the other does. The Challenge awarded 10 points for each victory in a round-robin tournament, with the score reduced based on the execution time of the winning solution.

The third-place solution by Eric Hangstefer used a purely offensive strategy, building bridges in an attempt to force the opponent to play defense. The other two solutions applied both offensive and defensive techniques, and between them won all of the games in the tournament. Gregory's solution plays defense whenever it believes that its opponent is closer to a solution than he is. To accomplish this, he maintains a Path data structure for himself and for his opponent identifying what he believes to be the best (shortest) path across the board for each player. A path score is maintained by counting the number of hexagons along the path that still need to be occupied to complete an unbroken chain (excluding "two-bridge" positions where a connection is guaranteed). One key to understanding the winning solution is the ExtendPathToEdge routine, used by the BestPathAcrossBoard routine to find a short route to each edge from a central hexagon.

CAPE AND TIGHTS NOT INCLUDED

Switch from reactive to PROACTIVE support and be the superhero your users expect



Search

Community Search:

MacTech Search:

Software Updates via MacUpdate

Remember when typing felt good?



Latest Forum Discussions

[SEE ALL](#)

iPHONE APP SHOWCASE

Price Scanner via MacPrices.net

Jobs Board

The tournament consisted of three test cases, with hex boards of size 8, 10, and 18, with each solution competing against each other twice, once playing first and once playing second. Gregory's solution and Jeff's solution competed against one another six times, and, interestingly enough, in all but one of those contests, the solution playing second (i.e., horizontal) won the game. In most of the games, Gregory's and Jeff's solutions required comparable execution time, except in two instances where Gregory's solution took very long to make its final move. This behavior accounted for the fact that the two solutions were very close in cumulative points, even though Gregory won two more games. By virtue of having the highest point total, the most victories, and (as a tie-breaker) the smallest code, Gregory's solution is the winner.

The final position in the 10x10 game where Gregory played the vertical orientation and won is reproduced below for your enjoyment. Note the vertical player's focus on a single path by comparison with the more scattered hexagons occupied by the horizontal player.

```
0 1 2 3 4 5 6 7 8 9
0 - - V H H - - - - H
1 - - V V - H - - H V
2 - - H V - - H - V -
3 - - V H H H - - - -
4 - - V - - V - V - -
5 - V H H - - - - -
6 - V V H H - V - - -
7 - - V V H - - V - -
8 - - H V - H - - - -
9 - - V H - - - - -
```

The table below lists for each entry the number of tournament wins, the total tournament points earned, and the code and data sizes. The number in parentheses after the entrant's name is the total number of Challenge points earned in all Challenges to date prior to this one.

Name	Wins	Points	Code	Data
Gregory Cooper (34)	10	80.08	8028	628
Jeff Mallett (64)	8	79.98	8608	392
Eric Hangstefer (2)	0	0.00	5236	88

Top 20 Contestants

Here are the Top Contestants for the Programmer's Challenge. The numbers below include points awarded over the 24 most recent contests, including points earned by this month's entrants.

Rank	Name	Points	Rank	Name	Points
1.	Munter, Ernst	184	11.	Cutts, Kevin	21
2.	Gregg, Xan	114	12.	Nicolle, Ludovic	21
3.	Larsson, Gustav	67	13.	Picao, Miguel Cruz	21
4.	Cooper, Greg	54	14.	Brown, Jorg	20
5.	Lengyel, Eric	40	15.	Gundrum, Eric	20
6.	Boring, Randy	37	16.	Higgins, Charles	20
7.	Mallett, Jeff	37	17.	Kasparian, Raffi	20
8.	Lewis, Peter	32	18.	Slezak, Ken	20
9.	Antoniewicz, Andy	24	19.	Studer, Thomas	20
10.	Beith, Gary	24	20.	Karsh, Bill	19

There are three ways to earn points: (1) scoring in the top 5 of any Challenge, (2) being the first person to find a bug in a published winning solution or, (3) being the first person to suggest a Challenge that I use. The points you can win are:

1st place	20 points
2nd place	10 points
3rd place	7 points
4th place	4 points
5th place	2 points
finding bug	2 points
suggesting Challenge	2 points

Here is Gregory's winning solution:

Hex_Strategy.c © 1997 Gregory Cooper

```
// *****
// General idea: My strategy tries to find the quickest routes across the
// board, both for itself and for its opponent. If the opponent has a better,
// shorter route than my strategy does, then my strategy makes a defensive move
// (one that makes it more difficult for the opponent to complete his route).
// Otherwise, it makes an offensive move (one which brings its own route closer
// to completion). It re-traces the routes during each turn, looking for
```

```

// necessary modifications and scoring changes.
// *****

#include "Storage.h"

// an enumeration for move types
typedef enum
{
    // counter-clockwise from right
    right,                // 0
    upRightTwoBridge,      // 30
    upRight,               // 60
    upTwoBridge,           // 90
    upLeft,                // 120
    upLeftTwoBridge,       // 150
    left,                  // 180
    downLeftTwoBridge,     // 210
    downLeft,              // 240
    downTwoBridge,         // 270
    downRight,             // 300
    downRightTwoBridge,    // 330
    noDirection             // undefined
} Direction;

// a data structure for representing potential routes to edges or to other chips
typedef struct Path
{
    long    row;           // row of piece on board
    long    col;           // column of piece on board
    Direction nextDirection; // direction to next piece
    Direction prevDirection; // direction to prev piece
    Boolean  occupied;      // location actually taken?
    Boolean  center;        // origin of path?
    struct Path *next;      // next piece in path
    struct Path *prev;      // previous piece in path
} Path;

// a data type for the playing chips
typedef enum {
    empty,
    blue,    // plays first
    red      // plays second
} Chip;

// an enumeration for disposing of unneeded paths
// (used by the BetterPath function and its clients)
typedef enum {
    leaveBad,    // indicates that inferior strategy should be left alone
    forgetBad    // indicates that inferior strategy should be disposed
} PathAction;

// an enumeration for the edges
typedef enum {
    leftEdge,
    rightEdge,
    topEdge,
    bottomEdge
} Edge;

// an enumeration for path disruptions
typedef enum {
    noDisruption,    // move did not block path
    inTwoBridge,     // move occupied part of a two-bridge
    inSpot           // move occupied a strategic location
} Disruption;

Function Prototypes
Path *BestPathAcrossBoard(long boardSize, Chip *theBoard,
    long startRow, long startCol, Chip color);
// tries to find a path across the board through (startRow, startCol)
// PRECONDITIONS: boardSize is the number of rows or columns
// on the board; theBoard stores information about the board;
// (startRow, startCol) indicates a piece of the given color on the board;
// color indicates which player for whom the path is to be found
// POSTCONDITIONS: returns a pointer to the path found, beginning with
// (startRow, startCol), or returns nil if no path exists

Boolean ExtendPathToEdge(long boardSize, Chip *theBoard,
    Path *path, Edge edge);
// tries to find a path from path to the given edge

```

```

// PRECONDITIONS: path is an incomplete path, edge is one of the four edges,
// boardSize is the number of rows or columns on the board, theBoard defines
// the playing board
// POSTCONDITIONS: if a path from the edge exists, it is found and the value
// true is returned; otherwise, all memory associated with the path is freed
// and it returns false

void PrioritizeMoves(Edge edge, Direction moveList[12]);
// determines the order in which potential moves in a path should be attempted
// PRECONDITIONS: edge is the edge to which we are trying to move
// POSTCONDITIONS: moveList contains the moves in order from most to
// least direct

Boolean PossibleMove(long boardSize, Chip *theBoard,
    long row, long col, Path *path, Direction move,
    Chip color);
// determines whether placing a piece in a given location is feasible
// PRECONDITIONS: boardSize is the number of rows or columns on the board;
// theBoard defines the board; (row, col) is the piece from which we are
// moving; move is the direction in which we are moving; color is the color of
// the player
// POSTCONDITIONS: returns true if the given move is possible, false otherwise

void FindNewLocation(long row, long col, Direction move,
    long *newRow, long *newCol);
// determines the location of a piece adjacent to another piece in a given
// direction
// PRECONDITIONS: (row, col) indicates the initial piece, move is the direction
// of the move
// POSTCONDITIONS: (*newRow, *newCol) contain the location of the new piece

Direction OppositeDirection(Direction move);
// determines the direction opposite a given direction
// PRECONDITIONS: move is a direction
// POSTCONDITIONS: returns the direction opposite move

Boolean OnBoard(long boardSize, long row, long col);
// determines whether a given piece lies on the board
// PRECONDITIONS: boardSize is the size of the board,
// (row, col) is the location of the desired piece
// POSTCONDITIONS: returns whether the piece is
// within the boundaries of the board

Boolean IsTwoBridge(Direction move);
// determines whether a connection in a given direction is a two-bridge
// PRECONDITIONS: move is the direction of the desired connection
// POSTCONDITIONS: returns true if the connection is a two-bridge;
// otherwise returns false

Boolean TwoBridgeFree(long boardSize, Chip *theBoard,
    long row, long col, Direction move);
// determines whether a given two-bridge is free
// PRECONDITIONS: (row, col) contains one of the pieces in in the two-bridge,
// move is the direction of the bridge, boardSize and theBoard define the board
// POSTCONDITIONS: returns true if the spaces in between are free;
// otherwise, returns false

Boolean PieceOnEdge(long row, long col, long boardSize,
    Edge edge);
// determines whether a given piece is on a desired edge
// PRECONDITIONS: (row, col) indicate a piece, boardSize is the number of rows
// or columns on the board, edge is the desired edge
// POSTCONDITIONS: returns true if the piece is on the edge;
// otherwise returns false

void FreePath(Path *path);
// disposes the memory associated with a path
// PRECONDITIONS: path points to a path
// POSTCONDITIONS: the memory associated with the path is freed.

Boolean ValidMove(long boardSize, Chip *theBoard,
    long row, long col);
// determines whether a desired move is legal
// PRECONDITIONS: boardSize contains the number of rows or columns on the
// board; theBoard defines the playing board; (row, col) is the location of the
// intended move
// POSTCONDITIONS: returns true if the piece is on the board and not already
// taken; otherwise false

```

```

Disruption MoveInPath(Path *path, long row, long col);
// determines whether a desired move is in a path
// PRECONDITIONS: path describes a path across the board;
// (row, col) defines the intended move
// POSTCONDITIONS: returns noDisruption if the move is not in the path,
// inTwoBridge if it threatens an established two-bridge, or inSpot if it
// occupies a location in the path (including a two-bridge whose end-points
// have not been secured)

Boolean TwoBridgeThreatened(long row, long col,
    long threatRow, long threatCol, Direction move);
// determines whether a two-bridge is threatened by a given move
// PRECONDITIONS: (row, col) is the origin of a two-bridge; (threatRow,
// threatCol) is the new move; move is the direction of the two-bridge
// POSTCONDITIONS: returns true if the two-bridge is threatened by the move

void UpdatePath(Path *path, long row, long col);
// updates a path when a move is made
// PRECONDITIONS: path points to a path across the board;
// (row, col) is the location of the move
// POSTCONDITIONS: the location in the path is marked as being occupied;
// sometimes, as when a two-bridge is filled in (and destroyed), an extra node
// is added to the path list

Direction GetDirection(long row1, long col1, long row2,
    long col2);
// determines the direction from one piece to another
// PRECONDITIONS: (row1, col1) and (row2, col2) define two pieces
// POSTCONDITIONS: the direction from (row1, col1) to (row2, col2) is returned

Path *BetterPath(Path *path1, Path *path2,
    PathAction action);
// determines which of two paths is superior to the other
// PRECONDITIONS: path1 and path2 point to paths
// POSTCONDITIONS: the better of the two paths is returned;
// if action is forgetBad, then the inferior path is disposed

long PathRating(Path *path);
// rates a path
// PRECONDITIONS: path is a path
// POSTCONDITIONS: a rating for the path is returned
// (always positive, low numbers are better)

void FillThreatenedTwoBridge(Path *path, long threatRow,
    long threatCol, long *row, long *col);
// determines where to move in order to secure a threatened two-bridge
// PRECONDITIONS: path points to a path;
// (threatRow, threatCol) is the move which threatened the two-bridge
// POSTCONDITIONS: (row, col) is the move needed to secure
// the free half of the two-bridge

Path *TakeDetour(long boardSize, Chip *theBoard, Path *path,
    long row, long col, Chip color);
// finds a new path after it has been disrupted
// PRECONDITIONS: path points to a path; (row, col)
// indicates a move which disrupted the path
// POSTCONDITIONS: the path is corrected, if possible, so that it avoids the
// opponent's piece, and it is then returned; nil is returned if no new path
// can be found

void FindBestMove(Path *ourPath, Path *oppPath, long *row,
    long *col);
// TRIES to determine the best move
// PRECONDITIONS: ourPath and oppPath are the expected
// paths of the player and the opponent, respectively
// POSTCONDITIONS: (*row, *col) indicate the chosen move, which should be
// decent; if the opponent's path is better, it should be a defensive move; if
// not, it should be an offensive move; whenever possible, offensive and
// defensive moves are
// made to coincide with one another

Boolean NextToEdge(long row, long col, long boardSize,
    Edge edge);
// determines if a piece is next to a given edge
// PRECONDITIONS: (row, col) is a piece; boardSize is the size of the board,
// edge is the edge we want to be near
// POSTCONDITIONS: returns true if the piece is one hex from the edge

Boolean Hex(long boardSize, long oppRow, long oppCol,

```

```
long *moveRow, long *moveCol, void *privStorage,  
Boolean newGame, Boolean playFirst);
```

Hex

```
Boolean Hex(long boardSize, long oppRow, long oppCol,  
long *moveRow, long *moveCol, void *privStorage,  
Boolean newGame, Boolean playFirst)  
{  
    // board storage (preserved between turns)  
    static Chip *theBoard;  
    // expected paths (also preserved between turns)  
    static Path *ourPath, *oppPath;  
    // a temporary path used for comparisons  
    Path *newPath;  
    Disruption disruption; // holds path disruptions  
  
    if (newGame)  
    {  
        long i;  
  
        // initialize the private storage allocation system  
        InitializeHeap(privStorage, 0x100000); // 1 MB  
        // initialize storage space for the board  
        theBoard = AllocateBlock(boardSize * boardSize);  
        for (i = 0; i < boardSize * boardSize; i++)  
            theBoard[i] = empty;  
        // indicate that no paths have yet been determined  
        ourPath = oppPath = nil;  
        if (playFirst)  
        {  
            // go in the middle  
            *moveRow = (boardSize - 1) / 2;  
            *moveCol = boardSize / 2;  
            // mark the chip on the board  
            theBoard[*moveRow * boardSize + *moveCol] =  
                blue; // blue because we are moving first  
            // try to determine the shortest route across the board through  
            // the spot where we just moved  
            ourPath = BestPathAcrossBoard(boardSize,  
                theBoard, *moveRow, *moveCol, blue);  
            return true;  
        } // end if  
    } // end if  
    // check the opponent's move  
    if (!ValidMove(boardSize, theBoard, oppRow, oppCol))  
        // if it is illegal, abort here  
        return false;  
    // mark the opponent's move on the board  
    theBoard[oppRow * boardSize + oppCol] =  
        playFirst ? red : blue;  
    // if the opponent's move was in his path (offensive)  
    if (MoveInPath(oppPath, oppRow, oppCol))  
        // update his path  
        UpdatePath(oppPath, oppRow, oppCol);  
    else // opponent did not continue his expected path  
    {  
        // try to find a probable route for the opponent through the piece  
        // where he just moved  
        newPath = BestPathAcrossBoard(boardSize, theBoard,  
            oppRow, oppCol, playFirst ? red : blue);  
        // if his new route is better than his old one, use it instead  
        oppPath = BetterPath(oppPath, newPath, forgetBad);  
    } // end else  
    // determine whether his move blocked or otherwise disrupted our path  
    disruption = MoveInPath(ourPath, oppRow, oppCol);  
    if (disruption == inTwoBridge)  
    {  
        // move into the other half of the  
        // two-bridge automatically  
        FillThreatenedTwoBridge(ourPath, oppRow, oppCol,  
            moveRow, moveCol);  
        // mark the move on the board  
        theBoard[*moveRow * boardSize + *moveCol] =  
            playFirst ? blue : red;  
        // update the path  
        UpdatePath(ourPath, *moveRow, *moveCol);  
    } // end if  
    else // no automatic response  
    {
```

```

if (disruption == inSpot)
    // try to find a new path from just
    // before the disruption (the unaffected
    // part of the path can remain)
    ourPath = TakeDetour(boardSize, theBoard,
        ourPath, oppRow, oppCol, playFirst ? blue :
        red);
// try to determine the best move and then make it
FindBestMove(ourPath, oppPath, moveRow, moveCol);
// if the move was invalid (happens when neither
// player has a path), just try to find a valid move
if (!ValidMove(boardSize, theBoard, *moveRow,
    *moveCol))
{
    Boolean moveFound = false;
    for (*moveRow = 0; !moveFound && *moveRow
        < boardSize; (*moveRow)++)
    {
        for (*moveCol = 0; !moveFound && *moveCol
            < boardSize; (*moveCol)++)
        {
            if (ValidMove(boardSize, theBoard,
                *moveRow, *moveCol))
            {
                moveFound = true;
                (*moveRow)--;
                (*moveCol)--;
            } // end if
        } // end for
    } // end for
} // end if
// mark the move on the board
theBoard[*moveRow * boardSize + *moveCol] =
    playFirst ? blue : red;
// check to see if the move was in our path
if (MoveInPath(ourPath, *moveRow, *moveCol))
    // if so, update it
    UpdatePath(ourPath, *moveRow, *moveCol);
else
{
    // otherwise, try to find a path through the
    // new location
    newPath = BestPathAcrossBoard(boardSize,
        theBoard, *moveRow, *moveCol,
        playFirst ? blue : red);

    // if the new path is better than the old one, use it instead
    ourPath = BetterPath(ourPath, newPath,
        forgetBad);
} // end else
} // end if
// if our move blocked the opponent's path,
// try to find him a new path from just before the disruption
if (MoveInPath(oppPath, *moveRow, *moveCol))
    oppPath = TakeDetour(boardSize, theBoard, oppPath,
        *moveRow, *moveCol, playFirst ? red : blue);
return true;
} // end Hex

```

BestPathAcrossBoard

```

Path *BestPathAcrossBoard(long boardSize, Chip *theBoard,
    long startRow, long startCol, Chip color)
{
    // initial path is just the starting hex
    Path *thePath = AllocateBlock(sizeof (Path));

    thePath->row = startRow;
    thePath->col = startCol;
    thePath->occupied =    // should be true always
        theBoard[startRow * boardSize + startCol] == color;
    // no connections yet
    thePath->prev = thePath->next = nil;
    thePath->prevDirection = thePath->nextDirection =
        noDirection;
    thePath->center = true;
    // find the best path from (startRow, startCol) to the
    // top (if blue) or left (if red) edge
    if (!ExtendPathToEdge(boardSize, theBoard, thePath,
        color == blue ? topEdge : leftEdge))

```

```

{
    FreePath(thePath);
    return nil;
} // end if
// find the best path from (startRow, startCol) to the
// other friendly edge
if (!ExtendPathToEdge(boardSize, theBoard, thePath,
    color == blue ? bottomEdge : rightEdge))
{
    FreePath(thePath);
    return nil;
} // end if
// if we made it here, we succeeded
return thePath;
} // end BestPathAcrossBoard

ExtendPathToEdge
Boolean ExtendPathToEdge(long boardSize, Chip *theBoard,
    Path *path, Edge edge)
{
    Direction moveList[12]; // stores the move priorities
    // which color we are
    Chip color = edge == leftEdge || edge == rightEdge ?
        red : blue;
    long moveNum; // an index for when we try to move
    // which way the path is going
    Boolean goingForward = edge == rightEdge ||
        edge == bottomEdge;
    Boolean result; // did we succeed yet

    // prioritize the moves according to which edge we
    // are seeking and the current location in the path
    PrioritizeMoves(edge, moveList);
    // stop if we are already at an edge
    if (PieceOnEdge(path->row, path->col, boardSize, edge))
        return true;
    // see if we are next to the edge
    if (NextToEdge(path->row, path->col, boardSize, edge))
    {
        Direction d;
        long newRow, newCol;

        // make the simplest move to reach the edge
        for (d = right; d < noDirection; d++)
            if (!IsTwoBridge(d))
            {
                FindNewLocation(path->row, path->col, d,
                    &newRow, &newCol);
                if (PieceOnEdge(newRow, newCol, boardSize,
                    edge) && ValidMove(boardSize, theBoard,
                    newRow, newCol))
                {
                    // allocate space for the new node
                    Path *newNode =
                        AllocateBlock(sizeof (Path));
                    newNode->row = newRow;
                    newNode->col = newCol;
                    // determine if it has been secured or not
                    newNode->occupied = theBoard[
                        newNode->row * boardSize +
                        newNode->col] == color;
                    // it is not the center (it is an extension)
                    newNode->center = false;
                    // link the new node to the old path
                    if (goingForward)
                    {
                        path->nextDirection = d;
                        path->next = newNode;
                        newNode->prevDirection =
                            OppositeDirection(d);
                        newNode->prev = path;
                        newNode->next = nil;
                        newNode->nextDirection =
                            noDirection;
                    } // end if
                    else
                    {
                        path->prevDirection = d;
                        path->prev = newNode;
                    }
                }
            }
    }
}

```

```

        newNode->nextDirection =
            OppositeDirection(d);
        newNode->next = path;
        newNode->prev = nil;
        newNode->prevDirection =
            noDirection;
    } // end else
    return true;
} // end if
} // end for
} // end if
// first look to connect with a piece that is already on the board
for (result = false, moveNum = 0;
    !result && moveNum < 9; moveNum++)
{
    // once a single move works, try to extend from
    // the new location to the same edge
    if (PossibleMove(boardSize, theBoard, path->row,
        path->col, path, moveList[moveNum], color))
    {
        // allocate space for the new node
        Path *newNode = AllocateBlock(sizeof (Path));

        // abort if there is not enough memory
        if (!newNode)
            return false;
        // define its position
        FindNewLocation(path->row, path->col,
            moveList[moveNum], &newNode->row,
            &newNode->col);
        // determine if it has been secured or not
        newNode->occupied = theBoard[newNode->row *
            boardSize + newNode->col] == color;
        // only make the move if there is already a connection
        if (!newNode->occupied)
        {
            FreeBlock(newNode);
            continue;
        }
        // it is not the center (it is an extension)
        newNode->center = false;
        // link the new node to the old path
        if (goingForward)
        {
            path->nextDirection = moveList[moveNum];
            path->next = newNode;
            newNode->prevDirection =
                OppositeDirection(moveList[moveNum]);
            newNode->prev = path;
            newNode->next = nil;
            newNode->nextDirection = noDirection;
        } // end if
        else
        {
            path->prevDirection = moveList[moveNum];
            path->prev = newNode;
            newNode->nextDirection =
                OppositeDirection(moveList[moveNum]);
            newNode->next = path;
            newNode->prev = nil;
            newNode->prevDirection = noDirection;
        } // end else
        // see if we can get to an edge
        if (ExtendPathToEdge(boardSize, theBoard,
            newNode, edge))
        {
            result = true;
        } // end if
        else
        {
            // undo the added node
            FreeBlock(newNode);
            if (goingForward)
            {
                path->next = nil;
                path->nextDirection = noDirection;
            } // end if
            else
            {

```

```

        path->prev = nil;
        path->prevDirection = noDirection;
    } // end else
} // end else
} // end if
} // end for
// try moves until we run out or one works
for (moveNum = 0; !result && moveNum < 7; moveNum++)
{
    // once a single move works, try to extend from
    // the new location to the same edge
    if (PossibleMove(boardSize, theBoard, path->row,
        path->col, path, moveList[moveNum], color))
    {
        // allocate space for the new node
        Path *newNode = AllocateBlock(sizeof (Path));

        // abort if there is not enough memory
        if (!newNode)
            return false;
        // define its position
        FindNewLocation(path->row, path->col,
            moveList[moveNum], &newNode->row,
            &newNode->col);
        // determine if it has been secured or not
        newNode-occupied = theBoard[newNode->row *
            boardSize + newNode->col] == color;
        // it is not the center (it is an extension)
        newNode-center = false;
        // link the new node to the old path
        if (goingForward)
        {
            path->nextDirection = moveList[moveNum];
            path->next = newNode;
            newNode->prevDirection =
                OppositeDirection(moveList[moveNum]);
            newNode->prev = path;
            newNode->next = nil;
            newNode->nextDirection = noDirection;
        } // end if
        else
        {
            path->prevDirection = moveList[moveNum];
            path->prev = newNode;
            newNode->nextDirection =
                OppositeDirection(moveList[moveNum]);
            newNode->next = path;
            newNode->prev = nil;
            newNode->prevDirection = noDirection;
        } // end else
        // see if we are at the edge or can get to one
        if (ExtendPathToEdge(boardSize, theBoard,
            newNode, edge))
        {
            result = true;
        } // end if
        else
        {
            // undo the added node
            FreeBlock(newNode);
            if (goingForward)
            {
                path->next = nil;
                path->nextDirection = noDirection;
            } // end if
            else
            {
                path->prev = nil;
                path->prevDirection = noDirection;
            } // end else
        } // end else
    } // end if
} // end for

// return true if we succeeded, false if not
return result;
} // end ExtendPathToEdge

```

PrioritizeMoves

```

void PrioritizeMoves(Edge edge, Direction move[12])
{
    switch (edge) {
        case leftEdge:
            // in order from most to least direct
            move[0] = downLeftTwoBridge;
            move[1] = upLeftTwoBridge;
            move[2] = downTwoBridge;
            move[3] = left;
            move[4] = downLeft;
            move[5] = upLeft;
            move[6] = downRight;
            move[7] = upRight;
            move[8] = right;
            move[9] = upTwoBridge;
            move[10] = downRightTwoBridge;
            move[11] = upRightTwoBridge;
            break;
        case rightEdge:
            // in order from most to least direct
            move[0] = upRightTwoBridge;
            move[1] = downRightTwoBridge;
            move[2] = upTwoBridge;
            // determines whether a desired move is in a path
    // PRECONDITIONS: path describes a path across the board;
    // (row, col) defines the intended move
    // POSTCONDITIONS: returns noDisruption if the move is not in the path,
    // inTwoBridge if it threatens an established two-bridge, or inSpot if it
    // occupies a location in the path (including a two-bridge whose end-points
    // have not been secured)

Boolean TwoBridgeThreatened(long row, long col,
    long threatRow, long threatCol, Direction move);
// determines whether a two-bridge is threatened by a given move
// PRECONDITIONS: (row, col) is the origin of a two-bridge; (threatRow, threatCol)
// is the new move; move is the direction of the two-bridge
// POSTCONDITIONS: returns true if the two-bridge is threatened by the move

void UpdatePath(Path *path, long row, long col);
// updates a path when a move is made
// PRECONDITIONS: path points to a path across the board;
// (row, col) is the location of the move
// POSTCONDITIONS: the location in the path is marked as being occupied;
// sometimes, as when a two-bridge is filled in (and destroyed), an extra node
// is added to the path list

Direction GetDirection(long row1, long col1, long row2,
    long col2);
// determines the direction from one piece to another
// PRECONDITIONS: (row1, col1) and (row2, col2) define two pieces
// POSTCONDITIONS: the direction from (row1, col1) to (row2, col2) is returned

Path *BetterPath(Path *path1, Path *path2,
    PathAction action);
// determines which of two paths is superior to the other
// PRECONDITIONS: path1 and path2 point to paths
// POSTCONDITIONS: the better of the two paths is returned;
// if action is forgetBad, then the inferior path is disposed

long PathRating(Path *path);
// rates a path
// PRECONDITIONS: path is a path
// POSTCONDITIONS: a rating for the path is returned
// (always positive, low numbers are better)

void FillThreatenedTwoBridge(Path *path, long threatRow,
    long threatCol, long *row, long *col);
// determines where to move in order to secure a threatened two-bridge
// PRECONDITIONS: path points to a path;
// (threatRow, threatCol) is the move which threatened the two-bridge
// POSTCONDITIONS: (row, col) is the move needed to secure
// the free half of the two-bridge

Path *TakeDetour(long boardSize, Chip *theBoard, Path *path,
    long row, long col, Chip color);
// finds a new path after it has been disrupted
// PRECONDITIONS: path points to a path; (row, col)
// indicates a move which disrupted the path
// POSTCONDITIONS: the path is corrected, if possible, so that it avoids the

```

```

// opponent's piece, and it is then returned; nil is returned if no new path
// can be found

void FindBestMove(Path *ourPath, Path *oppPath, long *row,
    long *col);
// TRIES to determine the best move
// PRECONDITIONS: ourPath and oppPath are the expected
// paths of the player and the opponent, respectively
// POSTCONDITIONS: (*row, *col) indicate the chosen move, which should be
// decent; if the opponent's path is better, it should be a defensive move; if
// not, it should be an offensive move; whenever possible, offensive and
// defensive moves are made to coincide with one another

Boolean NextToEdge(long row, long col, long boardSize,
    Edge edge);
// determines if a piece is next to a given edge
// PRECONDITIONS: (row, col) is a piece; boardSize is the size of the board,
// edge is the edge we want to be near
// POSTCONDITIONS: returns true if the piece is one hex from the edge

Boolean Hex(long boardSize, long oppRow, long oppCol,
    long *moveRow, long *moveCol, void *privStorage,
    Boolean newGame, Boolean playFirst);

Hex
Boolean Hex(long boardSize, long oppRow, long oppCol,
    long *moveRow, long *moveCol, void *privStorage,
    Boolean newGame, Boolean playFirst)
{
    // board storage (preserved between turns)
    static Chip *theBoard;
    // expected paths (also preserved between turns)
    static Path *ourPath, *oppPath;
    // a temporary path used for comparisons
    Path *newPath;
    Disruption disruption; // holds path disruptions

    if (newGame)
    {
        long i;

        // initialize the private storage allocation system
        InitializeHeap(privStorage, 0x100000); // 1 MB
        // initialize storage space for the board
        theBoard = AllocateBlock(boardSize * boardSize);
        for (i = 0; i < boardSize * boardSize; i++)
            theBoard[i] = empty;
        // indicate that no paths have yet been determined
        ourPath = oppPath = nil;
        if (playFirst)
        {
            // go in the middle
            *moveRow = (boardSize - 1) / 2;
            *moveCol = boardSize / 2;
            // mark the chip on the board
            theBoard[*moveRow * boardSize + *moveCol] =
                blue; // blue because we are moving first
            // try to determine the shortest route across the board through the
            // spot where we just moved
            ourPath = BestPathAcrossBoard(boardSize,
                theBoard, *moveRow, *moveCol, blue);
            return true;
        } // end if
    } // end if
    // check the opponent's move
    if (!ValidMove(boardSize, theBoard, oppRow, oppCol))
        // if it is illegal, abort here
        return false;
    // mark the opponent's move on the board
    theBoard[oppRow * boardSize + oppCol] =
        playFirst ? red : blue;
    // if the opponent's move was in his path (offensive)

    move[3] = right;
    move[4] = upRight;
    move[5] = downRight;
    move[6] = upLeft;
    move[7] = downLeft;
    move[8] = left;

```

```

        move[9] = downTwoBridge;
        move[10] = upLeftTwoBridge;
        move[11] = downLeftTwoBridge;
        break;
    case topEdge:
        // in order from most to least direct
        move[0] = upTwoBridge;
        move[1] = upLeftTwoBridge;
        move[2] = upRightTwoBridge;
        move[3] = upLeft;
        move[4] = upRight;
        move[5] = left;
        move[6] = right;
        move[7] = downLeft;
        move[8] = downRight;
        move[9] = downLeftTwoBridge;
        move[10] = downRightTwoBridge;
        move[11] = downTwoBridge;
        break;
    case bottomEdge:
        // in order from most to least direct
        move[0] = downTwoBridge;
        move[1] = downRightTwoBridge;
        move[2] = downLeftTwoBridge;
        move[3] = downRight;
        move[4] = downLeft;
        move[5] = right;
        move[6] = left;
        move[7] = upRight;
        move[8] = upLeft;
        move[9] = upRightTwoBridge;
        move[10] = upLeftTwoBridge;
        move[11] = upTwoBridge;
        break;
    default:
        break; // should never arrive here
} // end switch
} // end PrioritizeMoves

PossibleMove
Boolean PossibleMove(long boardSize, Chip *theBoard,
    long row, long col, Path *path, Direction move,
    Chip color)
{
    long newRow, newCol; // where the move leads

    // determine where a move in the new direction would lead
    FindNewLocation(row, col, move, &newRow, &newCol);
    // the new location must be on the board and either
    // empty or occupied by a friendly piece; it must also
    // not be in the path already
    if (!OnBoard(boardSize, newRow, newCol)
        || (theBoard[newRow * boardSize + newCol] ==
            (color == blue ? red : blue)) ||
        MoveInPath(path, newRow, newCol))
        return false;
    // if the move forms a two-bridge, there must
    // be no pieces in between the two
    if (IsTwoBridge(move))
    {
        return TwoBridgeFree(boardSize, theBoard, row, col,
            move);
    } // end if
    else
        return true;
} // end PossibleMove

FindNewLocation
void FindNewLocation(long row, long col, Direction move,
    long *newRow, long *newCol)
{
    // handle the row first
    switch (move) {
        case upTwoBridge:
            *newRow = row - 2;
            break;
        case upRightTwoBridge:
        case upRight:
        case upLeft:

```

```

        case upLeftTwoBridge:
            *newRow = row - 1;
            break;
        case left:
        case right:
            *newRow = row;
            break;
        case downLeftTwoBridge:
        case downLeft:
        case downRight:
        case downRightTwoBridge:
            *newRow = row + 1;
            break;
        case downTwoBridge:
            *newRow = row + 2;
            break;
        default: // should never get here
            break;
    } // end switch
    // then handle the column
    switch (move) {
        case downLeftTwoBridge:
            *newCol = col - 2;
            break;
        case upLeftTwoBridge:
        case left:
        case downLeft:
        case downTwoBridge:
            *newCol = col - 1;
            break;
        case upLeft:
        case downRight:
            *newCol = col;
            break;
        case upTwoBridge:
        case upRight:
        case right:
        case downRightTwoBridge:
            *newCol = col + 1;
            break;
        case upRightTwoBridge:
            *newCol = col + 2;
            break;
        default: // should never get here
            break;
    } // end switch
} // end FindNewLocation

OppositeDirection
Direction OppositeDirection(Direction move)
{
    switch (move) {
        case upTwoBridge:
            return downTwoBridge;
        case upRightTwoBridge:
            return downLeftTwoBridge;
        case upRight:
            return downLeft;
        case upLeft:
            return downRight;
        case upLeftTwoBridge:
            return downRightTwoBridge;
        case left:
            return right;
        case right:
            return left;
        case downLeftTwoBridge:
            return upRightTwoBridge;
        case downLeft:
            return upRight;
        case downRight:
            return upLeft;
        case downRightTwoBridge:
            return upLeftTwoBridge;
        case downTwoBridge:
            return upTwoBridge;
        default: // should never get here
            break;
    } // end switch
}

```

```

} // end OppositeDirection

OnBoard
Boolean OnBoard(long boardSize, long row, long col)
{
    return row == 0 && row < boardSize && col == 0 &&
        col < boardSize;
} // end OnBoard

IsTwoBridge
Boolean IsTwoBridge(Direction move)
{
    switch (move) {
        case upTwoBridge:
        case upRightTwoBridge:
        case upLeftTwoBridge:
        case downLeftTwoBridge:
        case downRightTwoBridge:
        case downTwoBridge:
            return true;
        default:
            return false;
    } // end switch
} // end IsTwoBridge

TwoBridgeFree
Boolean TwoBridgeFree(long boardSize, Chip *theBoard,
    long row, long col, Direction move)
{
    switch (move) {
        case upTwoBridge:
            return theBoard[(row - 1) * boardSize + col] ==
                empty && theBoard[(row - 1) * boardSize +
                    col + 1] == empty;
        case upRightTwoBridge:
            return theBoard[(row - 1) * boardSize +
                col + 1] == empty && theBoard[row *
                    boardSize + col + 1] == empty;
        case upLeftTwoBridge:
            return theBoard[(row - 1) * boardSize +
                col] == empty && theBoard[row *
                    boardSize + col - 1] == empty;
        case downLeftTwoBridge:
            return theBoard[(row + 1) * boardSize +
                col - 1] == empty && theBoard[row *
                    boardSize + col - 1] == empty;
        case downRightTwoBridge:
            return theBoard[(row + 1) * boardSize +
                col] == empty && theBoard[row *
                    boardSize + col + 1] == empty;
        case downTwoBridge:
            return theBoard[(row + 1) * boardSize +
                col] == empty && theBoard[(row + 1) *
                    boardSize + col - 1] == empty;
        default:
            return false;
    } // end switch
} // TwoBridgeFree

PieceOnEdge
Boolean PieceOnEdge(long row, long col, long boardSize,
    Edge edge)
{
    switch (edge) {
        case leftEdge:
            return col == 0;
        case rightEdge:
            return col == boardSize - 1;
        case topEdge:
            return row == 0;
        case bottomEdge:
            return row == boardSize - 1;
        default: // should never get here
            break;
    } // end switch
} // end PieceOnEdge

FreePath
void FreePath(Path *path)

```

```

{
    if (!path)
        return;
    // go to the beginning of the path
    for (; path-prev; path = path-prev);
    // free each node until none are left
    for (; path; path = path-next)
        FreeBlock(path);
} // end FreePath

ValidMove
Boolean ValidMove(long boardSize, Chip *theBoard,
    long row, long col)
{
    return OnBoard(boardSize, row, col) &&
        theBoard[row * boardSize + col] == empty; // free
} // end ValidMove

MoveInPath
Disruption MoveInPath(Path *path, long row, long col)
{
    if (!path)
        return noDisruption;

    // go to the beginning of the path
    for (; path-prev; path = path-prev);
    // search the path until the end is reached or a node
    // is found matching the desired move
    for (; path; path = path-next)
        // check for a threatened two-bridge
        if (IsTwoBridge(path-nextDirection) &&
            TwoBridgeThreatened(path-row, path-col,
                row, col, path-nextDirection))
        {
            if (path-occupied && path-next-occupied)
                return inTwoBridge;
            else
                return inSpot;
        } // end if
    else if (row == path-row && col == path-col)
        return inSpot;
    // if we get here, there are no disruptions
    return noDisruption;
} // end MoveInPath

TwoBridgeThreatened
Boolean TwoBridgeThreatened(long row, long col,
    long threatRow, long threatCol, Direction move)
{
    switch (move) {
        case upTwoBridge:
            return (threatRow == row - 1) &&
                (threatCol == col || threatCol == col + 1);
        case upRightTwoBridge:
            return (threatCol == col + 1) &&
                (threatRow == row - 1 || threatRow == row);
        case upLeftTwoBridge:
            return (threatRow + threatCol == row + col - 1)
                && (threatRow == row || threatCol == col);
        case downLeftTwoBridge:
            return (threatCol == col - 1) &&
                (threatRow == row || threatRow == row + 1);
        case downRightTwoBridge:
            return (threatRow + threatCol == row + col + 1)
                && (threatRow == row || threatCol == col);
        case downTwoBridge:
            return (threatRow == row + 1) &&
                (threatCol == col || threatCol == col - 1);
        default: // should not get here
            return false;
    } // end switch
} // end TwoBridgeThreatened

UpdatePath
void UpdatePath(Path *path, long row, long col)
{
    // go to the beginning of the path
    for (; path-prev; path = path-prev);
    // search the path until the end is reached or a node

```

```

// is found matching the desired move
for (; path; path = path-next)
    // check for completion of a two-bridge
    if (IsTwoBridge(path-nextDirection) &&
        TwoBridgeThreatened(path-row, path-col,
            row, col, path-nextDirection))
    {
        // destroy the two-bridge and create two new connections
        Path *newNode = AllocateBlock(sizeof (Path));

        newNode-row = row;
        newNode-col = col;
        newNode-occupied = true;
        newNode-center = false;
        newNode-prevDirection = GetDirection(
            row, col, path-row, path-col);
        newNode-nextDirection = GetDirection(
            row, col, path-next-row, path-next-col);
        newNode-prev = path;
        newNode-next = path-next;
        path-nextDirection = OppositeDirection(
            newNode-prevDirection);
        path-next = newNode;
        newNode-next-prevDirection =
            OppositeDirection(newNode-nextDirection);
        newNode-next-prev = newNode;

    } // end if
    else if (row == path-row && col == path-col)
        path-occupied = true;
} // end UpdatePath

GetDirection
Direction GetDirection(long row1, long col1, long row2,
    long col2)
{
    // check for unique cases first
    if (row2 == row1 + 2)
        return downTwoBridge;
    else if (row2 == row1 - 2)
        return upTwoBridge;
    else if (col2 == col1 + 2)
        return upRightTwoBridge;
    else if (col2 == col1 - 2)
        return downLeftTwoBridge;
    // then try the more common ones
    else if (row2 == row1 - 1)
        if (col2 == col1 - 1)
            return upLeftTwoBridge;
        else if (col2 == col1)
            return upLeft;
        else // col2 == col1
            return upRight;
    else if (row2 == row1 + 1)
        if (col2 == col1 - 1)
            return downLeft;
        else if (col2 == col1 + 1)
            return downRightTwoBridge;
        else // col2 == col1
            return downRight;
    else // row2 == row1
        if (col2 == col1 - 1)
            return left;
        else // col2 == col1 + 1
            return right;
} // end GetDirection

BetterPath
Path *BetterPath(Path *path1, Path *path2,
    PathAction action)
{
    long rating1, rating2;
    Path *betterPath, *worsePath;

    // rate both paths
    rating1 = PathRating(path1);
    rating2 = PathRating(path2);
    // compare the ratings (low is better)
    if (rating1 < rating2)

```

```

{
    betterPath = path1;
    worsePath = path2;
} // end if
else
{
    betterPath = path2;
    worsePath = path1;
} // end else
// dispose of the bad path, if directed to do so
if (action == forgetBad)
    FreePath(worsePath);
return betterPath;
} // end BetterPath

PathRating
long PathRating(Path *path)
{
    long rating;

    if (!path)
        return 4096;
    // go to the beginning of the path
    for (; path-prev; path = path-prev);
    // count the number of unoccupied locations
    for (rating = 0; path; path = path-next)
        if (path-occupied == false)
            rating++;
    return rating;
} // end PathRating

FillThreatenedTwoBridge
void FillThreatenedTwoBridge(Path *path, long threatRow,
    long threatCol, long *row, long *col)
{
    // go to the beginning of the path
    for (; path-prev; path = path-prev);
    // scan through the path until the threat is found
    for (; path; path = path-next)
        if (IsTwoBridge(path-nextDirection) &&
            TwoBridgeThreatened(path-row, path-col,
                threatRow, threatCol, path-nextDirection))
        {
            // try all moves until we find one that fills
            // the two-bridge (sorry, it's the easiest way
            // way to do it)
            Direction d;
            Boolean done;

            for (done = false, d = 0; !done &&
                d < noDirection; d++)
            {
                FindNewLocation(path-row, path-col,
                    d, row, col);
                // make sure we're not making the same move
                // as the opponent
                if (TwoBridgeThreatened(path-row,
                    path-col, *row, *col,
                    path-nextDirection) && (*row !=
                        threatRow || *col != threatCol))
                {
                    done = true;
                } // end if
            } // end for
        } // end if
} // end FillThreatenedTwoBridge

TakeDetour
Path *TakeDetour(long boardSize, Chip *theBoard, Path *path,
    long row, long col, Chip color)
{
    Boolean centerPassed; // have we passed the center?
    Boolean blockFound; // have we located the disruption?
    Path *corruptPath; // the part that was blocked
    Edge edge;

    // go to the beginning of the path
    for (; path-prev; path = path-prev);
    // find the location of the disruption

```

```

for (centerPassed = blockFound = false; path &&
    !blockFound;)
{
    if (path-center)
        centerPassed = true;

    if (IsTwoBridge(path-nextDirection) &&
        TwoBridgeThreatened(path-row, path-col,
            row, col, path-nextDirection))
    {
        if (centerPassed)
            path = path-next;
        blockFound = true;
    } // end if
    else if (row == path-row && col == path-col)
        blockFound = true;
    else
        path = path-next;
} // end for
// free from there to the end
if (centerPassed)
{
    corruptPath = path;
    path = path-prev;
    path-next = corruptPath-prev = nil;
    FreePath(corruptPath);
} // end if
else
{
    corruptPath = path;
    path = path-next;
    corruptPath-next = path-prev = nil;
    FreePath(corruptPath);
} // end else
if (color == blue)
    if (centerPassed)
        edge = bottomEdge;
    else
        edge = topEdge;
else // color is red
    if (centerPassed)
        edge = rightEdge;
    else
        edge = leftEdge;
// extend the path from there to the edge
if (!ExtendPathToEdge(boardSize, theBoard, path, edge))
{
    // if not possible, destroy the path
    FreePath(path);
    path = nil;
} // end if

return path;
} // end TakeDetour

FindBestMove
void FindBestMove(Path *ourPath, Path *oppPath, long *row,
    long *col)
{
    // ratings for our path, the opponent's path, and the
    // two halves of the better (used) path
    long ourRating, oppRating, prevRating = 0,
        nextRating = 0;
    Path *usedPath, *tempPath, *unusedPath;
    // don't even try to continue if there are no paths
    if (!oppPath && !ourPath)
        return;
    // see who has the better path
    ourRating = PathRating(ourPath);
    oppRating = PathRating(oppPath);
    if (ourRating < oppRating) // ours is better
    {
        usedPath = ourPath;
        unusedPath = oppPath;
    }
    else
    {
        usedPath = oppPath;
        unusedPath = ourPath;
    }
}

```

```

}
// first try to make a move which is both offensive and defensive
tempPath = usedPath;
// go to the beginning
for (; tempPath-prev; tempPath = tempPath-prev);
// find the first unoccupied piece
for (; tempPath && tempPath-occupied;
    tempPath = tempPath-prev)
{
    // see if it is in the other path, also
    if (MoveInPath(usedPath, tempPath-row,
        tempPath-col))
    {
        *row = tempPath-row;
        *col = tempPath-col;
        return;
    }
}
// look for unoccupied pieces
tempPath = usedPath;
// go to the beginning
for (; tempPath-prev; tempPath = tempPath-prev);
// go to the center
for (; !tempPath-center; tempPath = tempPath-next);
// rate the first half
for (; tempPath; tempPath = tempPath-prev)
    prevRating += !tempPath-occupied;
tempPath = usedPath;
// go to the beginning
for (; tempPath-prev; tempPath = tempPath-prev);
// go back to the center
for (; !tempPath-center; tempPath = tempPath-next);
// rate the second half
for (; tempPath; tempPath = tempPath-next)
    nextRating += !tempPath-occupied;
tempPath = usedPath;
// go to the beginning
for (; tempPath-prev; tempPath = tempPath-prev);
// go to the center
for (; !tempPath-center; tempPath = tempPath-next);
// make a move on the side with the worse (high) rating
if (prevRating > nextRating)
    // find the first unoccupied piece
    for (; tempPath && tempPath-occupied;
        tempPath = tempPath-prev);
else
    // find the first unoccupied piece
    for (; tempPath && tempPath-occupied;
        tempPath = tempPath-next);
if (tempPath)
{
    *row = tempPath-row;
    *col = tempPath-col;
} // end if
else // no unoccupied pieces anywhere
{
    // if there are none, fill in any unconnected two-bridges
    for (; usedPath-prev; usedPath =
        usedPath-prev);
    for (; usedPath && !IsTwoBridge(
        usedPath-nextDirection);
        usedPath = usedPath-next);
    if (usedPath) // should always be true
    {
        // try all moves until we find one that
        // fills the two-bridge (sorry, it's the
        // easiest way way to do it)
        Direction d;
        Boolean done;

        for (done = false, d = 0; !done &&
            d < noDirection; d++)
        {
            FindNewLocation(usedPath-row,
                usedPath-col, d, row, col);
            if (TwoBridgeThreatened(usedPath-row,
                usedPath-col, *row, *col,
                usedPath-nextDirection))
            {

```

```

        done = true;
    } // end if
} // end for
} // end if
} // end else
} // end FindBestMove

NextToEdge
Boolean NextToEdge(long row, long col, long boardSize,
    Edge edge)
{
    switch (edge) {
        case leftEdge:
            return col == 1;
        case rightEdge:
            return col == boardSize - 2;
        case topEdge:
            return row == 1;
        case bottomEdge:
            return row == boardSize - 2;
        default: // should never get here
            break;
    } // end switch
} // end NextToEdge

Dynamic_Memory4.c
typedef struct Header
{
    long
        size;    // the size in bytes of the block

    struct Header
        *prev,    // the previous block's header
        *next;    // the next block's header
} Header;

void InitializeHeap(Header *heapStart, long size);
void ChopBlock(Header *theBlock, long fragSize);
void *AllocateBlock(long size);
void FuseBlocks(Header *block);
void FreeBlock(void *data);
void HeapSummary(void);
Boolean IsPointerValid(void *data);
void SetHeap(Header *start);
Header *GetHeap(void);

Header
    *gHeapStart;

long
    gFreeBlocks,
    gUsedBlocks,
    gFreeSpace,
    gUsedSpace,
    gTotalSpace;

void InitializeHeap
(
    // these are assumed to be valid; no error-checking is performed
    Header *heapStart,    // the address of the start of the heap zone
    long size    // the size of the zone
)
{
    heapStart-size = size - sizeof (Header);
                                // mark the size of the zone
    heapStart-prev = nil;    // mark as being the first block...
    heapStart-next = nil;    // ...and the last
    gHeapStart = heapStart;    // initialize the global heap start pointer
}

void ChopBlock
(
    // these are assumed to be valid; fragSize must be less than theBlock-size
    Header *theBlock,    // the block to chop
    long fragSize    // the size of the first new fragment
)
{
    Header
        *newBlock;

    newBlock = (Header *) (((char *) theBlock) + sizeof (Header) + fragSize);

```

```

newBlock-size = theBlock-size - fragSize - sizeof (Header);
                // the remaining space
newBlock-prev = theBlock; // newBlock comes after theBlock
newBlock-next = theBlock-next;
                // it gets inserted between theBlock and
                // theBlock-next

if (theBlock-next)
    theBlock-next-prev = newBlock;
theBlock-size = fragSize; // theBlock gets just what was asked for
theBlock-next = newBlock; // its new next block is newBlock
}

void *AllocateBlock
(
    long    size    // the size of the desired block
)
{
    const long
        BLOCK_IN_USE_MASK = 0x80000000, // if this bit is set in the 'size'
                                         // field, the block is in use
        MIN_BLOCK_SIZE = 80; // the smallest usable block (board data + pointers)

    Header
        *currentBlock; // a pointer to the block currently being considered

    currentBlock = gHeapStart;

    // loop until either there are no more blocks or a fitting one is found
    while (currentBlock && ((currentBlock-size &
        BLOCK_IN_USE_MASK) || (currentBlock-size < size)))
        currentBlock = currentBlock-next;

    if (currentBlock) // a valid block was found
    {
        // if the block is much bigger than needed
        if ((currentBlock-size - size)
            (MIN_BLOCK_SIZE + sizeof (Header)))
            ChopBlock(currentBlock, size); // cut off a piece of exactly the
            currentBlock-size |= BLOCK_IN_USE_MASK; // mark as being in use

        // return a pointer to just after the block header
        return (char *) (currentBlock + 1);
    }
    else
        // indicate that allocation failed
        return nil;
}

void FuseBlocks
(
    Header *block // the block to fuse with its following neighbor
)
{
    Header
        *neighbor;

    neighbor = block-next;
    block-next = neighbor-next;
    neighbor-next-prev = block;
    block-size += sizeof (Header) + neighbor-size;
    // blocks must not be in use
}

void FreeBlock
(
    void
        *data // the block of data to free, fusing with its neighbors if possible
)
{
    const long
        BLOCK_IN_USE_MASK = 0x80000000;
        // bit in size field used to indicate use of blocks

    Header
        *theBlock; // the data block's header
    // the header is located just before the data
    theBlock = ((Header *) data) - 1;

    // assumed to be in use, with flag set; here it is reset

```

// right

```

    if (theBlock-size & BLOCK_IN_USE_MASK)
        theBlock-size -= BLOCK_IN_USE_MASK;

    // if there is a next block and it is free
    if (theBlock-next &&
        !(theBlock-next-size & BLOCK_IN_USE_MASK))
        FuseBlocks(theBlock);

    // if there is a previous block and it is free
    if (theBlock-prev &&
        !(theBlock-prev-size & BLOCK_IN_USE_MASK))
        FuseBlocks(theBlock-prev);
}

void HeapSummary
(
    void
)
{
    const long
        IN_USE_MASK = 0x80000000;

    Header
        *block;

    block = gHeapStart;
    while (block-next)
        block = block-next;

    gFreeBlocks = gUsedBlocks = gFreeSpace = gUsedSpace = 0;

    while (block)
    {
        if (block-size & IN_USE_MASK)
        {
            gUsedBlocks++;
            gUsedSpace += block-size - IN_USE_MASK;
        }
        else
        {
            gFreeBlocks++;
            gFreeSpace += block-size;
        }
        block = block-prev;
    }
    gTotalSpace = (gFreeBlocks + gUsedBlocks) *
        sizeof (Header) + gFreeSpace + gUsedSpace;
}

Boolean IsPointerValid(void *data)
{
    Header
        *theBlock,
        *compBlock;

    theBlock = ((Header *) data) - 1;
    compBlock = gHeapStart;

    while(compBlock && compBlock != theBlock)
        compBlock = compBlock-next;

    if (compBlock == theBlock)
        return true;
    else
        return false;
}

void SetHeap(Header *start)
{
    gHeapStart = start;
}

Header *GetHeap(void)
{
    return gHeapStart;
}

Storage.h
typedef struct Header

```

```

{
    long
        size,          // the size in bytes of the block
        inUse;

    struct Header
        *prev,          // the previous block's header
        *next,          // the next block's header
        *prevFree,      // the previous free block
        *nextFree;      // the next free block
} Header;

void AddToFreeList(Header *block);
void RemoveFromFreeList(Header *block);
void InitializeHeap(Header *heapStart, long size);
void ChopBlock(Header *theBlock, long fragSize);
void *AllocateBlock(long size);
void FuseBlocks(Header *block);
void FreeBlock(void *data);
void SetHeap(Header *block);
void SetFreeStart(Header *freeStart);
Header *GetFreeStart(void);

```

SPREAD THE WORD: [!\[\]\(0bfc65f44c8abd2b170640998d4b431b_img.jpg\) Slashdot](#) [!\[\]\(aa7167e86baaa00341f89a22d70208d7_img.jpg\) Digg](#) [!\[\]\(a4c5e18638254f4952c9d5a00e9e5613_img.jpg\) Del.icio.us](#) [!\[\]\(9c86a2a0c0f022b66d3c09e1a96c6895_img.jpg\) Reddit](#) [!\[\]\(2c89e97f680381aceb1f1b29086f20f8_img.jpg\) Newsvine](#)

Generate a short URL for this page: [GET LINK](#)

MacTech Magazine. www.mactech.com

Toll Free 877-MACTECH, Outside US/Canada: 805-494-9797

MacTech is a registered trademark of Xplain Corporation. Xplain, "The journal of Apple technology", Apple Expo, Explain It, MacDev, MacDev-1, THINK Reference, NetProfessional, Apple Expo, MacTech Central, MacTech Domains, MacNews, MacForge, and the MacTutorMan are trademarks or service marks of Xplain Corporation. Sprocket is a registered trademark of eSprocket Corporation. Other trademarks and copyrights appearing in this printing or software remain the property of their respective holders. Not responsible for typographical errors.

All contents are Copyright 1984-2011 by Xplain Corporation. All rights reserved. Theme designed by [Icreon](#).